Data Modeling & ETL

From raw CSV to a queryable star schema: a hands-on pipeline with PostgreSQL and NYC Yellow Taxi data

Module 2 · Data Engineering | Lecture 1

Agenda

What we will cover today

1	**Data platforms**	Where data lives: databases, warehouses, lakes, lakehouses
2	**Data modeling**	How to structure data so it can be queried reliably
3	**ETL pipelines**	How data moves from source to warehouse, step by step
4	**Hands-on repo**	Docker PostgreSQL, Python ingestion, star schema in SQL

The problem

Real-world data is fragmented

Across services. Across time. Across third parties.

- Apps & APIs: every microservice writes to its own database
- Third-party services: payment providers, CRMs, ad platforms, each with its own schema
- Logs & events: clickstreams and sensors, streaming continuously
- Manual uploads: spreadsheets, partner drops, government datasets



Storage architectures

	Primary use	Examples	Strengths	Weakness
Database	Operational transactions	PostgreSQL, MySQL	Fast writes	Poor for analytics
Data warehouse	Analytics & BI	BigQuery, Snowflake	Structured queries	Costly at scale
Data lake	Raw, unstructured data	S3, HDFS	Flexible, cheap	Hard governance
Lakehouse	Unified analytics layer	Databricks	Combines both worlds	New, complex

Different systems hold different states of the data. The source of truth depends on the purpose: a dashboard reads from the data warehouse.

Data lake vs data warehouse

	Data lake	Data warehouse
Data format	Raw, any format	Cleaned, structured
Schema	Schema-on-read	Schema-on-write
Users	Data engineers, ML engineers	Analysts, BI tools
Query speed	Slower (raw data)	Fast (optimized)
Cost	Cheap storage	More expensive compute
When to use	Exploration, ML training data, RAG corpora	Dashboards, reporting, analytics

Many organizations use both: a lake for raw storage, a warehouse for analytics.

Data modeling: design the structure first

Like designing classes before writing code.

Conceptual

High-level entities and relationships.

"Customers place orders for products."

Logical

Entities, attributes, keys, relationships.

Technology-agnostic.

Physical

Actual tables, columns, indexes.

PostgreSQL, BigQuery, etc.

Data modeling

Normalization: reduce redundancy and anomalies

Problem table: multi-value cells, repeated names, and a category that depends on the course name, not the key.

StudentID	StudentName	CourseID	CourseName	Category
101	Alice Hope	C1, C2	Physics, Math	Science
102	Bob Vance	C1	Physics	Science

1 1NF

One value per cell. Split multi-value cells into separate rows.

2 2NF

Remove partial dependencies. Every non-key attribute depends on the whole key.

3 3NF

Remove transitive dependencies. Each fact is stored exactly once.

Normalized vs dimensional models

Normalized (operational)

- Reduces redundancy
- Safer updates and consistency
- Represents relationships precisely
- Tradeoff: analytical queries need many joins

Dimensional (analytical)

- One central fact table with measurable events
- Surrounding dimension tables give context
- Reporting queries are easier to read
- Tradeoff: some redundancy by design

Both are valid. They serve different workloads.

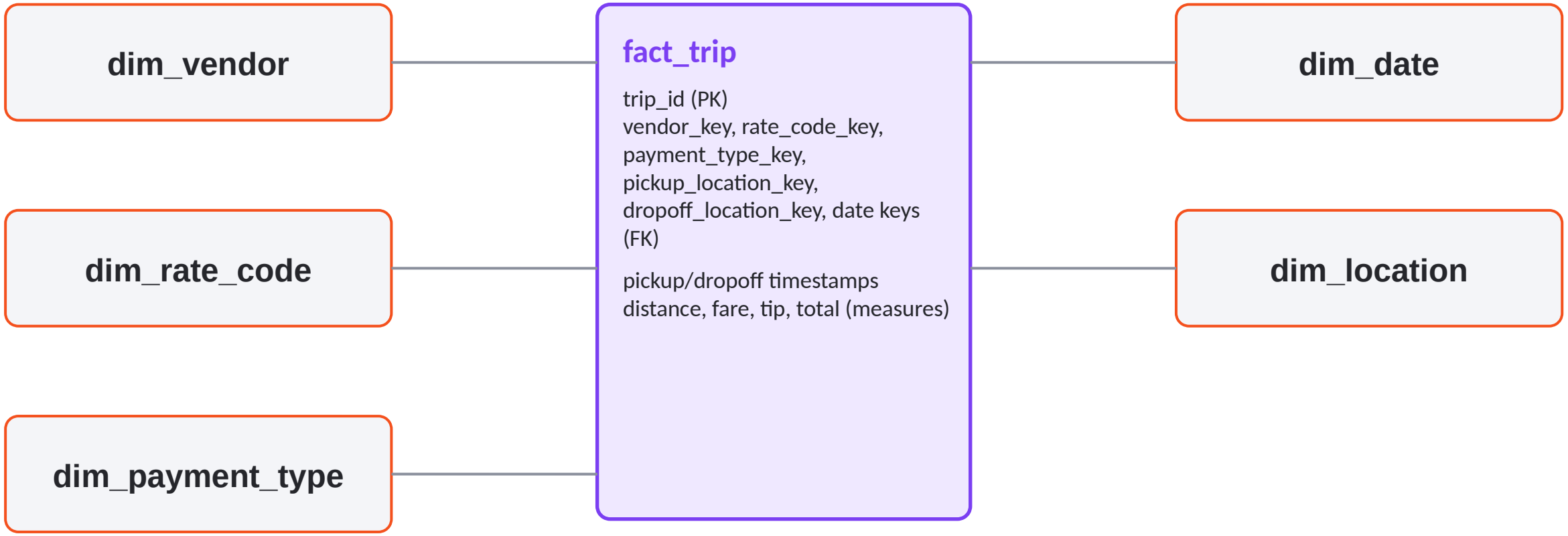
Step 1: choose the grain

What should one row in our fact table represent?

One row per taxi trip

- Trip amounts and distances become measures in the fact table
- Calendar data becomes a date dimension
- Source codes (vendor, rate code, payment type, location) become reusable dimensions
- Analysts can group by vendor, payment type, location and date without redefining the business event

The star schema we will build



Dimensions: descriptive context

Fact: measurable events

Fact tables vs dimension tables

	Fact tables	Dimension tables
Contains	Numeric measurements (revenue, count, quantity)	Descriptive attributes (name, category, location)
Grain	One row per event or transaction	One row per entity
Keys	Foreign keys referencing dimension tables	Primary key referenced by the fact table
Changes	Grows continuously (append-only)	Changes slowly (updated in place)

ETL: Extract, Transform, Load

Extract

Pull raw data from sources: APIs, databases, files, third parties.

Transform

Clean, validate, reshape. Apply business logic. Standardize formats.

Load

Write to the target: warehouse, database or data lake.

ETL transforms before loading.
ELT loads raw data first and transforms inside the warehouse (next session, with dbt). In practice, most organizations use a blend of both.

Hands-on repo

The pipeline we build in this repository

1

Start PostgreSQL

postgres:17 in Docker on the ny-taxi network

2

Download Parquet

January 2025 Yellow Taxi file, reused if present

3

Inspect with pandas

Check columns and types before loading

4

Create yellow_taxi

Table schema from the Parquet header

5

Append in batches

100,000 rows per batch, chunked for memory

6

Validate row counts

Database count must match the Parquet count

7

Model the star schema

Staging view, dimensions, fact_trip

Step 2: a staging view cleans names once

```
CREATE OR REPLACE VIEW yellow_taxi_clean AS
SELECT
  "VendorID" AS vendor_key,
  CASE WHEN "RatecodeID" IS NULL THEN NULL
       ELSE "RatecodeID"::INTEGER END AS rate_code_key,
  payment_type AS payment_type_key,
  "PULocationID" AS pickup_location_key,
  "DOLocationID" AS dropoff_location_key,
  tpep_pickup_datetime AS pickup_at,
  tpep_dropoff_datetime AS dropoff_at,
  ...  -- pickup_date, dropoff_date, trip_duration_minutes
```

- Renames mixed-case source columns into clear snake_case names
- Adds pickup_date and dropoff_date for date-dimension joins
- Calculates trip_duration_minutes once, so later SQL stays short
- Later cells read from this view, not from the raw table

Step 3: load dimensions first, then the fact table

1

Build dimensions

dim_date, dim_vendor, dim_rate_code, dim_payment_type, dim_location

2

Load fact_trip

One row per trip, with foreign keys pointing at the dimensions

3

Validate

fact_trip must have the same number of rows as yellow_taxi

Dimensions first, because the fact table's foreign keys must point at rows that already exist.

Reliability

Pipelines fail quietly unless you design against it

Risk	What happens	Guard in this repo
Schema drift	Upstream adds or renames a column; transforms break	Check column names and types before transforming
Silent failure	Half the data loads and no error is raised	Count rows after load and compare with the source
Duplicates	A rerun doubles the row count	Table is recreated before appending (replace, then append)
Broken assumptions	Dates arrive as DD/MM/YYYY, not YYYY-MM-DD	Parse and standardize types in the transform step

Rule of thumb: if you cannot re-run it and get the same result, it is not a pipeline.

Hands-on repo

The ingestion script: one CLI, also run in Docker

```
python data_ingestion.py \
  --url <parquet-url> \
  --file_path data/yellow_tripdata.parquet \
  --host localhost --port 5432 \
  --db ny_taxi --table_name yellow_taxi
```

- --url and --file_path are required
- Defaults: postgres user and password, localhost:5432, ny_taxi, yellow_taxi
- Same logic runs in the notebook and in the Docker image (uv run)

Inside load_parquet_to_postgres

1. Read the Parquet schema from the first batch and create or replace the table
2. Append batches of 100,000 rows, logging progress
3. Run SELECT COUNT(*) and report the database row count

Checkpoint

You are done when...

Step 1: Database

- ny-taxi-db container is running
- PostgreSQL answers on localhost:5432
- You can connect to ny_taxi with psql

Step 2: ETL

- yellow_taxi table exists in ny_taxi
- Notebook validation: DB row count equals Parquet row count
- The CLI and Docker flow make sense

Step 3: Model

- Dimension tables and fact_trip exist
- fact_trip row count equals yellow_taxi
- You can answer the notebook exercises

Review

Key takeaways and self-check questions

Review

Key takeaways

Data platforms

Database for transactions, warehouse for analytics, lake for raw data, lakehouse for both.

Modeling levels

Conceptual (entities), logical (keys, relationships), physical (tables, indexes).

Normalization

1NF, 2NF, 3NF remove redundancy. Good for storage, but more joins to query.

Star schema

Choose the grain first. One fact table of events, surrounded by dimensions.

ETL

Extract, transform, load. ELT loads raw first and transforms in the warehouse.

Reliability

Check schemas, count rows, avoid duplicates, and make reruns idempotent.

Review

Self-check questions

Q1 Why does the fact table need to be loaded after the dimensions?

Q2 What is the grain of fact_trip, and why does choosing it come first?

Q3 Why does a rerun of the ingestion script not double the row count?

Q4 What is the difference between ETL and ELT, and when would you pick each?

Review

Answers

Q1 Foreign keys in fact_trip must point to dimension rows that already exist.

Q2 One row per taxi trip. It fixes what measures and dimensions mean before any table is built.

Q3 The table is recreated (replace) before the batches are appended, so each run starts from a clean table.

Q4 ETL transforms before loading, suited to curated data. ELT loads raw data first and transforms in the warehouse, suited to flexible, self-service analysis.

Thank you!

Questions and discussion are welcome.